BadCourt — Rewrite Implementation Plan

Project: BadCourt — Badminton Court Booking Platform
Supersedes: `se121.badcourt` (.NET 9 microservices + Flutter + Next.js)
Date: 2 September 2026

---

1. Context & Decisions

This plan governs a full-stack rewrite of the `se121.badcourt` project. The following decisions were settled before planning:

Decision	Choice	Rationale
Goal	Pragmatic product	Optimise for shipping and low maintenance burden
Scope	Full rewrite	API, Angular admin panel, Flutter mobile app
Hosting	Local / demo only	Docker Compose or .NET Aspire on a single machine
Team	Solo, no deadline	Favours fewer moving parts over parallel workstreams
Architecture	Modular monolith	7 modules, one deployable, one database
Backend patterns	Clean Architecture + CQRS	Retained for extensibility and read/write separation
API surface	Controllers	Not Minimal APIs
Admin web	Angular	Not Next.js

Why a modular monolith

The predecessor's difficulty was distribution tax, not domain complexity:

Old symptom	Root cause	Resolution
`UserLockedConsumer` duplicated in 4 services	`UserState` copied into 4 datastores	One `users` table; join it
`AdminService` + `ManagerService` exist only to fan out HTTP calls	Data split across services	Dashboards become SQL
`Order` carries 8 denormalised fields	Cannot join across service databases	Join
`AdminDashboardController` duplicated in 3 services	No single query surface	One `Analytics` module
`SharedKernel` referenced by all 10 services	Distributed monolith already	Honest monolith
63 Ocelot routes to maintain	Gateway needed to reassemble data	No gateway

Approximately 50 project files collapse to 27.

---

2. Module Boundaries

One deployable, one PostgreSQL database, one schema per module.

Module	Responsibility	Replaces
Identity	Users, roles, JWT + refresh, photos, lock/unlock, presence	`AuthService`
Facilities	Registration, approval workflow, geo location, opening hours	`FacilityService`
Courts	Courts, pricing, inactive periods, availability	`CourtService`
Booking	Orders, Stripe, state machine, cancellation, ratings	`OrderService`
Social	Posts, comments, likes, reports	`PostService`
Messaging	Groups, messages, notifications, SignalR, email	`RealtimeService` + `EmailService`
Analytics	Admin and manager dashboards (read-only)	`AdminService` + `ManagerService`

3. Architecture at a Glance

Both roles currently target the same database. Introducing a read replica requires changing only the read connection string.

---

4. Solution Layout

```
BadCourt/
+-- BadCourt.slnx
+-- Directory.Packages.props           <- central package versions
+-- Directory.Build.props              <- nullable, warnings-as-errors, analyzers
+-- .editorconfig
+-- docker-compose.yml                 <- lab demo fallback
+-- src/
|   +-- api/
|   |   +-- BadCourt.AppHost/          <- Aspire: Postgres, MinIO, Mailpit, Stripe CLI
|   |   +-- BadCourt.ServiceDefaults/  <- OTel, health checks, resilience
|   |   +-- BadCourt.Api/              <- host: DI, middleware, hubs, OpenAPI
|   |   +-- BadCourt.SharedKernel/     <- Result, Error, Entity, dispatcher abstractions
|   |   +-- BadCourt.Integration/      <- cross-module contracts + integration events
|   |   +-- modules/
|   |       +-- Identity/    { Domain, Application, Infrastructure, Presentation }
|   |       +-- Facilities/  { Domain, Application, Infrastructure, Presentation }
|   |       +-- Courts/      { Domain, Application, Infrastructure, Presentation }
|   |       +-- Booking/     { Domain, Application, Infrastructure, Presentation }
|   |       +-- Social/      { Domain, Application, Infrastructure, Presentation }
|   |       +-- Messaging/   { Domain, Application, Infrastructure, Presentation }
|   |       +-- Analytics/   { Application, Infrastructure, Presentation }
|   +-- admin/                         <- Angular
|   +-- mobile/                        <- Flutter
+-- tests/
    +-- BadCourt.IntegrationTests/
    +-- BadCourt.UnitTests/
    +-- BadCourt.ArchitectureTests/
```

Each module drops the predecessor's `Infrastructure.Services` and `Infrastructure.Configuration` into a single `Infrastructure` project. `Analytics` is read-only and therefore has no `Domain` project.

Shared projects

- **`SharedKernel`** — pure primitives: `Result<T>`, `Error`, `Entity`, `AggregateRoot`, `IDomainEvent`, `PagedList<T>`, handler interfaces. Knows nothing about the domain.
- **`Integration`** — the only place modules learn about each other: cross-module interfaces (e.g. `ICourtPricingService`) and integration events (e.g. `FacilityApprovedEvent`). Replaces the old `SharedKernel` that held all 30 DTOs.

Reference rules (enforced by architecture tests)

```
Domain         -> SharedKernel only
Application    -> Domain, SharedKernel, Integration
Infrastructure -> Application, Domain, SharedKernel, Integration
Presentation   -> Application, SharedKernel
Api            -> all Presentation + all Infrastructure (composition root only)
```

No module may reference another module's `Domain`, `Application` or `Infrastructure`. Cross-module traffic goes through `Integration`.

---

5. The CQRS Read/Write Split

Handler abstractions

```
public interface ICommand<TResponse>;
public interface IQuery<TResponse>;

public interface ICommandHandler<in TCommand, TResponse>
    where TCommand : ICommand<TResponse>
{
    Task<Result<TResponse>> Handle(TCommand command, CancellationToken ct);
}

public interface IQueryHandler<in TQuery, TResponse>
    where TQuery : IQuery<TResponse>
{
    Task<Result<TResponse>> Handle(TQuery query, CancellationToken ct);
}
```

Pipeline behaviours without MediatR

MediatR moved to commercial licensing during 2025. Scrutor (MIT) provides equivalent pipeline behaviour through decorators:

```
services.Scan(s => s.FromAssemblies(ApplicationAssemblies)
    .AddClasses(c => c.AssignableTo(typeof(ICommandHandler<,>)))
    .AsImplementedInterfaces().WithScopedLifetime());

services.Decorate(typeof(ICommandHandler<,>), typeof(ValidationDecorator<,>));
services.Decorate(typeof(ICommandHandler<,>), typeof(TransactionDecorator<,>));
services.Decorate(typeof(ICommandHandler<,>), typeof(LoggingDecorator<,>));

// Queries get validation + logging only - no transaction, no domain events
services.Decorate(typeof(IQueryHandler<,>), typeof(ValidationDecorator<,>));
```

Decorator order matters: logging outermost, then transaction, then validation innermost.

The read connection

```
public interface IReadConnectionFactory
{
    Task<NpgsqlConnection> OpenAsync(CancellationToken ct);
}

internal sealed class ReadConnectionFactory(IOptions<DatabaseOptions> opt) : IReadConnectionFactory
{
    public async Task<NpgsqlConnection> OpenAsync(CancellationToken ct)
    {
        var conn = new NpgsqlConnection(opt.Value.ReadConnection);
        await conn.OpenAsync(ct);
        return conn;
    }
}
```

```
"Database": {
  "WriteConnection": "Host=localhost;Database=badcourt;Username=badcourt_app;...",
  "ReadConnection":  "Host=localhost;Database=badcourt;Username=badcourt_read;...;Application
Name=badcourt-read"
}
```

Adding a replica later changes only the second string; Npgsql routes natively:

```
Host=primary,replica1,replica2;Target Session Attributes=prefer-standby
```

Enforce read-only at the database

```
CREATE ROLE badcourt_read LOGIN PASSWORD '...';
GRANT CONNECT ON DATABASE badcourt TO badcourt_read;
GRANT USAGE ON SCHEMA identity, facilities, courts, booking, social, messaging TO badcourt_read;
GRANT SELECT ON ALL TABLES IN SCHEMA identity, facilities, courts, booking, social, messaging TO
badcourt_read;
ALTER DEFAULT PRIVILEGES IN SCHEMA identity, facilities, courts, booking, social, messaging
    GRANT SELECT ON TABLES TO badcourt_read;
ALTER ROLE badcourt_read SET default_transaction_read_only = on;
```

The final statement causes any write attempted through the read path to fail at the database, during development. Architecture tests additionally assert that no `IQueryHandler` implementation references a `WriteDbContext`.

Read technology selection

Read shape	Tool	Rationale
Lists, filters, pagination, dashboards	**Dapper** + hand-written SQL	Aggregates such as `RevenueByHour` and `ProvinceRevenue` are awkward in LINQ and fast in SQL
Simple by-id / detail reads	`ReadDbContext` (EF, `NoTracking`, no migrations)	Less SQL to maintain for the routine majority

Both bind to `ReadConnection`. Queries return DTOs directly and never domain entities, so no mapping library is required on the read side.

The write side remains classic: command to handler, load aggregate via repository, invoke domain method, `SaveChangesAsync`, then dispatch domain events within the same transaction.

---

6. Domain Model Improvements

6.1 Make double-booking impossible in the database

The predecessor enforced booking conflicts in application code (`IsTimePeriodInside`, `/orders/check-conflict`), which is racy under concurrency. PostgreSQL can enforce it as an invariant:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE booking.bookings
  ADD CONSTRAINT bookings_no_overlap
  EXCLUDE USING gist (court_id WITH =, period WITH &&)
  WHERE (status <> 'Cancelled');
```

`period` is a `tstzrange`. EF Core cannot model this, so it is added via a raw-SQL migration. Concurrent bookings for the same slot fail with a constraint violation mapped to `409 Conflict`.

6.2 Remove denormalisation

`Order` copied `Username`, `UserImageUrl`, `FacilityName`, `CourtName`, `Province` and `Address` solely because joins across service databases were impossible. Retain exactly two snapshots — **price at time of booking** and **facility name for receipts** — as genuine historical facts. Join for everything else.

6.3 Real geospatial search

`Facility.Location` plus the computed `Distance` field become a PostGIS `geography(Point,4326)` column with a GiST index, queried through NetTopologySuite.

6.4 Type consistency

`Court.State` was a `string` while `Facility.State` was an enum; make both enums. Use `DateTimeOffset` / `timestamptz` throughout, replacing the mixture of `DateTime.UtcNow` and `TimeOnly`. Give `Money` an explicit currency.

6.5 Explicit state machines

`OrderState` transitions were scattered across handlers and two background services. Model them as aggregate methods that reject invalid transitions.

> **Retain the event vocabulary.** `FacilityApprovedEvent`, `OrderCancelledEvent`, `CourtInactiveUpdatedEvent` and the rest are well-named and map cleanly onto in-process domain events. The business rules encoded in the predecessor's 25 consumers are its most valuable asset.

---

7. Backend Stack

Concern	Choice
Runtime	.NET 10 (LTS)
API	ASP.NET Core Controllers, `[ApiController]`, per-module `ApplicationPart`
Dispatch	Hand-rolled `ISender` + Scrutor decorators
Write data	EF Core 10 + Npgsql, one `WriteDbContext` per module, schema-per-module
Read data	Dapper + `ReadDbContext` (NoTracking) on `badcourt_read`
Database	PostgreSQL 17 + PostGIS + `btree_gist`
Validation	FluentValidation (via `ValidationDecorator`)
Mapping	Mapperly (write side only; reads project directly)
Auth	ASP.NET Core Identity + JWT + refresh tokens
Realtime	SignalR in-process, 2 hubs
Events	In-process dispatcher, same transaction
Jobs	`BackgroundService` + `PeriodicTimer`
Cache	HybridCache (in-memory)
Files	`IFileStorage` to MinIO
Email	MailKit to Mailpit (dev)
Payments	Stripe.net + Stripe CLI
Errors	`Result<T>` + `ProblemDetails` via `IExceptionHandler`
Docs	Built-in OpenAPI + Scalar
Local orchestration	.NET Aspire
Observability	OpenTelemetry + Serilog

8. Angular Admin Stack

Concern	Choice	Notes
Framework	Angular v20+, standalone components, no NgModules	`ng new admin --style=scss --ssr=false`
State	Signals (`signal` , `computed` , `resource`) in injectable services	NgRx SignalStore only if it outgrows this
Async data	`httpResource()` / `resource()`	Replaces the hand-rolled `useState` / `useEffect` of the old admin
UI	PrimeNG	Strongest data-table support; the admin has five heavy tables
Styling	Tailwind v4 alongside PrimeNG themes	
Charts	ngx-echarts	Revenue-by-hour, by-region and monthly charts
Forms	Typed Reactive Forms	Built in; no external form or validation library needed
HTTP	`HttpClient` + functional interceptors	One auth interceptor, one 401-refresh interceptor, one error interceptor
API client	`ng-openapi-gen` from the backend OpenAPI document	Eliminates the 13 hardcoded `localhost:{1000,4000,5000}` URLs
Auth	JWT in memory + refresh token in httpOnly cookie; `CanActivateFn` guards	Structurally immune to the predecessor's `userData` bypass
Realtime	`@microsoft/signalr` wrapped in a signal-exposing service	
Testing	Vitest + Playwright	Angular's Karma path is deprecated

The old admin's authentication bypass originated in NextAuth's `authorize` callback trusting client-supplied input. A SPA consuming server-issued JWTs has no equivalent seam, because the browser never asserts its own roles.

---

9. Flutter Mobile Stack

Concern	Choice	Fixes
State	Riverpod	Removes `BuildContext` from services, which blocked all testing
Routing	go_router	Replaces the 31-case `onGenerateRoute` switch; adds auth redirect guards
HTTP	Dio + interceptors	Replaces 51 hand-built `Bearer` headers; single 401-refresh location
Models	freezed + json_serializable	Replaces 24 hand-written `fromJson` implementations
Token storage	flutter_secure_storage	Replaces plaintext `SharedPreferences`
Config	`--dart-define-from-file`	Stops shipping `.env` as an extractable asset
Realtime	`signalr_netcore` wrapped to expose Streams	Replaces manual `setCallbacks` wiring
Lints	`flutter_lints` in `dev_dependencies`	The mis-indented dependency hid 333 deprecations
Logging	`logger`, stripped in release	Replaces 578 `print()` calls

Structure feature-first with `data / domain / presentation` per feature. Apply a soft 400-line ceiling per file; the predecessor's `facility_registration_screen.dart` reached 2,003 lines.

---

10. Implementation Phases

#	Phase	Key deliverables	Exit criteria
0	Foundation	Solution, CPM, `.gitignore`, SharedKernel (`Result`, `Entity`, handler interfaces), Scrutor dispatcher + 3 decorators, Aspire AppHost (Postgres + PostGIS + MinIO + Mailpit), both DB roles, ServiceDefaults, GitHub Actions, first architecture test	`dotnet test` green; Aspire dashboard shows a healthy app
1	Identity	4 projects; Identity + JWT + refresh; login, signup, PIN verify, reset password; photos; lock/unlock; role seeding	Login, refresh and a protected endpoint work end to end. This module is the template every other module copies.
2	Facilities	Registration, approval/rejection workflow, PostGIS geo search, opening hours, photos; `FacilityApprovedEvent`	Register, admin approves, event fires — proven by integration test
3	Courts	CRUD, pricing, inactive periods, availability query on the read path	Availability query executes on `badcourt_read`
4	**Booking**	`tstzrange` + GiST exclusion constraint; Stripe intent + webhook; order state machine; two background jobs; ratings	Concurrency test: 20 parallel bookings for one slot yield exactly 1 success and 19 `409`s
5	Social	Posts, comments, likes, reports, moderation	Feed and moderation flows covered by integration tests
6	Messaging	`ChatHub` + `AppHub`, groups, messages, notifications, presence, email via Mailpit	Two browser tabs exchange messages live
7	Analytics	Read-only module: SQL views + Dapper for all admin and manager dashboards	Module holds no write-context reference
8	Angular admin	Scaffold, generated API client, auth + guards, 7 feature areas, SignalR, Playwright e2e for the approval flow	Admin approves a facility end to end
9	Flutter app	Scaffold, Dio + interceptors, generated models, auth, player flows, manager flows, realtime	Book a court and pay with a Stripe test card
10	Hardening	Seed/demo data, OTel dashboards, `EXPLAIN` on slow queries, README and demo script	One command brings up a demoable system

Phases 1 and 4 carry the material risk. Phase 1 establishes the pattern repeated six times, so the module template must be correct before proceeding; an error there costs six refactors. Phase 4 is the actual product.

---

11. Conventions and Guardrails

- **Secrets** — `dotnet user-secrets` locally; Aspire parameters for container credentials. Nothing in `appsettings*.json`. The predecessor leaked a JWT signing key, a Cloudinary secret and a Gmail app password this way.
- **Migrations** — per module, into its own schema, with a `__EFMigrationsHistory_{module}` table. The exclusion constraint requires a raw-SQL migration.
- **Errors** — handlers return `Result<T>`; `ResultExtensions.ToActionResult()` maps to `ProblemDetails`. No exceptions for control flow; the predecessor used 21 exception types for routing.
- **Time** — `DateTimeOffset` and `timestamptz` everywhere. No `DateTime.UtcNow` in domain code; inject `TimeProvider` so booking logic is testable.
- **Architecture tests from day one** — assert no cross-module references, that `Domain` references nothing, that no `IQueryHandler` touches a write context, and that all controllers live in `Presentation`.

Testing strategy

The predecessor had zero backend tests and a 2-byte `widget_test.dart`.

- xUnit v3 + Testcontainers — real PostgreSQL per test run, no mocking of the data layer
- `WebApplicationFactory` integration tests per controller
- Unit tests for genuine logic: booking conflicts, pricing, availability windows, order state transitions
- Architecture tests for module boundaries
- GitHub Actions: build, test, `dotnet format --verify-no-changes`, `ng lint`, `flutter analyze`

Coverage percentage is not a target. Booking conflicts, authentication and facility approval are.

---

12. Immediate Prerequisites

1. **Rotate the credentials leaked by the predecessor**, independent of this rewrite. The JWT signing key, Cloudinary secret and Gmail app password are in git history and permit minting admin tokens for any user.
2. **Initialise the new repository with** `dotnet new gitignore` and use `dotnet user-secrets` for local configuration. The predecessor's `.gitignore` contained a single line (`.codegpt`), which is how the above were committed.

End of document.